Part 02 of midterm project:

3.1 Task 1.1: Start Part 2 Environment (1 pt)

```
#16 resolving provenance for metadata file
#16 DONE 0.0s
[+] up 12/12
 ✓ Image part02-etl-service          Built
 ✓ Network part02_midterm_network    Created
 ✓ Volume part02_warehouse_data      Created
 ✓ Volume part02_sales_shard_se_data Created
 ✓ Volume part02_sales_shard_ne_data Created
 ✓ Volume part02_hr_data             Created
 ✓ Container midterm_sales_shard_ne  Healthy
 ✓ Container midterm_sales_shard_se  Healthy
 ✓ Container midterm_hr_db           Healthy
 ✓ Container midterm_warehouse_db    Healthy
 ✓ Container midterm_pgadmin         Created
 ✓ Container midterm_etl             Created
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02>
```

Now all the six services are created and healthy.

Task 1.2: Count Rows in Each Shard (1 pt)

Southeast shard:

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec sales-shard-se psql -U student -d sales -c 'SELECT COUNT(*)
FROM customers;'
>> docker compose exec sales-shard-se psql -U student -d sales -c 'SELECT COUNT(*) FROM orders;'
>> docker compose exec sales-shard-se psql -U student -d sales -c 'SELECT COUNT(*) FROM products;'
 count
-------
     7
(1 row)

 count
-------
    24
(1 row)

 count
-------
    10
(1 row)
```

Northeast Shard:

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec sales-shard-ne psql -U student -d sales -c 'SELECT COUNT(*)
FROM customers;'
>> docker compose exec sales-shard-ne psql -U student -d sales -c 'SELECT COUNT(*) FROM orders;'
>> docker compose exec sales-shard-ne psql -U student -d sales -c 'SELECT COUNT(*) FROM products;'
 count
-------
     1
(1 row)

 count
-------
     3
(1 row)

 count
-------
    10
(1 row)

PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02>
```

Task 1.3: Analyze Uneven Distribution (2 pts):

The observed uneven distribution between the **Southeast and Northeast shards** highlights a classic issue in sharded database design. When one shard contains significantly more data (7 customers and ~24 orders) than another (1 customer and ~3 orders), it leads to poor query load balancing. Most queries will be directed toward the Southeast shard, causing it to become a hotspot, while the Northeast shard remains underutilized. This imbalance can degrade performance, increase query latency, and limit the system's ability to scale efficiently.

From a storage perspective, uneven distribution results in inefficient resource utilization. One shard consumes more disk space and processing power, while others remain idle. This defeats the purpose of horizontal scaling, where data and workload should be evenly distributed across nodes.

Additionally, the impact of a single shard failure becomes more severe. If the overloaded Southeast shard fails, a large portion of the system's data and query workload becomes unavailable, significantly affecting system reliability.

A better shard key would be one with higher cardinality and uniform distribution, such as customer_id (using hashing) or order_id. These keys distribute data more evenly across shards, ensuring balanced query load, efficient storage usage, and improved fault tolerance.

Part 2: Update ETL Pipeline (8 points)

Task 2.1: Diagnose the ETL Failure (2 pts)

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec etl-service cat /app/etl_script.py
service "etl-service" is not running
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose logs etl-service
midterm_etl  | exec ./entrypoint.sh: no such file or directory
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02>
```

The ETL script fails in Part 2 because it still uses the function get_connection("SALES"), which is designed to connect to a single database using environment variables such as SALES_DB_HOST and SALES_DB_PORT. However, in Part 2, the architecture has changed to a sharded system with two separate databases: sales-shard-se and sales-shard-ne, configured using environment variables like SALES_SHARD_SE_DB_HOST and SALES_SHARD_NE_DB_HOST. As a result, the existing connection logic cannot locate the correct database and fails.

The lines in the script where get_connection("SALES") is called must be modified to establish two separate connections—one for each shard. The extraction logic must also be updated to query both shards independently and combine the results in the application layer. This ensures that data from both shards is included in the ETL process, restoring correct functionality in the distributed database setup.

Task 2.2: Modify the ETL Script (4 pts)

etl_script.py 2, M ● Dockerfile M

midterm > part02 > etl > etl_script.py > run_etl

```python
def run_etl():
    print("=" * 60)

    # --- EXTRACT ---

    print("\n[EXTRACT] Reading from operational databases...")
    '''
    sales_conn = get_connection("SALES")
    customers, products, orders = extract_sales(sales_conn)
    sales_conn.close()
    '''
    # Connect to both shards
    sales_conn_se = get_connection("SALES_SHARD_SE")
    sales_conn_ne = get_connection("SALES_SHARD_NE")

    # Extract from both shards
    customers_se, products_se, orders_se = extract_sales(sales_conn_se)
    customers_ne, products_ne, orders_ne = extract_sales(sales_conn_ne)

    # Merge data
    customers = customers_se + customers_ne
    products = products_se   # same reference data, use one shard only
    orders = orders_se + orders_ne

    # Close connections
    sales_conn_se.close()
    sales_conn_ne.close()
```

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose build etl-service
#1 [internal] load local bake definitions
#1 reading from stdin 649B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 469B 0.0s done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 ...

#4 [auth] library/python:pull token for registry-1.docker.io
#4 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 0.7s

#5 [internal] load .dockerignore
#5 transferring context: 2B done
#5 DONE 0.0s

#6 [1/8] FROM docker.io/library/python:3.11-slim@sha256:9358444059ed78e2975ada2c189f1c1a3144a5dab6f35bff8c981afb38946634
#6 resolve docker.io/library/python:3.11-slim@sha256:9358444059ed78e2975ada2c189f1c1a3144a5dab6f35bff8c981afb38946634 0.0s
 done
#6 DONE 0.0s

#7 [internal] load build context
#7 transferring context: 94B done
#7 DONE 0.0s

#8 [4/8] WORKDIR /app
#8 CACHED
```

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose build etl-service
 ✓ Image part02-etl-service Built                                                                          4.4s
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose up -d etl-service
[+] up 6/6
 ✓ Network part02_midterm_network   Created                                                               0.1ss
 ✓ Container midterm_warehouse_db   Healthy                                                               11.5s
 ✓ Container midterm_sales_shard_ne Healthy                                                               11.0s
 ✓ Container midterm_sales_shard_se Healthy                                                               11.0s
 ✓ Container midterm_hr_db          Healthy                                                               11.0s
 ✓ Container midterm_etl            Created                                                               0.1ss
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose logs -f etl-service
midterm_etl  | === ETL Service Starting ===
midterm_etl  | Running initial ETL load...
midterm_etl  | ============================================================
midterm_etl  | ETL Pipeline Starting
midterm_etl  | ============================================================
midterm_etl  |
midterm_etl  | [EXTRACT] Reading from operational databases...
midterm_etl  |   Extracted 7 customers from sales-db
midterm_etl  |   Extracted 10 products from sales-db
midterm_etl  |   Extracted 24 orders from sales-db
midterm_etl  |   Extracted 1 customers from sales-db
midterm_etl  |   Extracted 10 products from sales-db
midterm_etl  |   Extracted 3 orders from sales-db
midterm_etl  |   Extracted 12 employees from hr-db
midterm_etl  |
midterm_etl  | [TRANSFORM] Preparing dimension data...
midterm_etl  |   Generated 365 dim_date rows for 2026
midterm_etl  |   Employee dimension: denormalized with department info (done in extract query)
midterm_etl  |
midterm_etl  | [LOAD] Writing to data warehouse...
midterm_etl  |   Loaded dim_date (365 rows, conflicts skipped)
midterm_etl  |   Loaded dim_customer (8 rows, upserted)
```

The ETL script in Part 1 used a single connection (get_connection("SALES")) to extract data from one database. In Part 2, the system is sharded into two databases: sales-shard-se and sales-shard-ne. To support this, the ETL script was modified to create two connections using get_connection("SALES_SHARD_SE") and get_connection("SALES_SHARD_NE"). Data extraction was performed separately for each shard. Customers and orders were combined from both shards, as they represent distinct records distributed across nodes. Products were extracted from only one shard since they are identical reference data. The merged datasets were then passed into the existing transform and load phases without modification. This ensures that the ETL pipeline correctly aggregates distributed data and loads a complete dataset into the warehouse.

Task 2.3: Verify Warehouse Data (2 pts)

Step 1 – Count verification

```
warehouse=# SELECT 'dim_customer' AS table_name, COUNT(*) FROM dim_customer
UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product
UNION ALL
SELECT 'fact_sales', COUNT(*) FROM fact_sales;
  table_name  | count
--------------+-------
 dim_customer |     8
 dim_product  |    10
 fact_sales   |    27
(3 rows)

warehouse=#
```

Step 2 – Insert a new order on the Southeast shard

```
sales=# -- Run on sales-shard-se
INSERT INTO orders (customer_id, product_id, quantity, total_price, order_date)
VALUES (2, 1, 1, 1299.99, '2026-02-25');
INSERT 0 1
```

Step 3 – Trigger the ETL manually:

```
warehouse=# \q
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec sales-shard-se psql
 -U student -d sales
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.
```

Step 4 – Verify the new order appears in the warehouse:

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec etl-service python
/app/etl_script.py
[LOAD] Writing to data warehouse...
  Loaded dim_date (365 rows, conflicts skipped)
  Loaded dim_customer (8 rows, upserted)
  Loaded dim_product (10 rows, upserted)
  Loaded dim_employee (12 rows, upserted)
  Loaded fact_sales (1 inserted, 27 already existed)

============================================================
ETL Pipeline Complete
============================================================
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec warehouse-db psql -
U student -d warehouse
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.

warehouse=# SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
 sale_key | full_date  | first_name | last_name | product_name  | quantity | total_price
----------+------------+------------+-----------+---------------+----------+-------------
       52 | 2026-02-25 | Bob        | Smith     | Laptop Pro 15 |        1 |     1299.99
       27 | 2026-02-14 | Eva        | Davis     | Laptop Pro 15 |        1 |     1299.99
       26 | 2026-01-28 | Eva        | Davis     | Standing Desk |        1 |      599.99
(3 rows)

warehouse=#
```

The count verification confirms that the fact_sales table increased by one record after inserting a new order into the shard database and triggering the ETL process. The query joining fact and dimension tables shows the newly inserted order appearing with correct attributes such as date, customer, product, and total price. This validates that the ETL pipeline successfully extracts data from distributed shards, transforms it into the star schema format, and loads it into the data warehouse.

Part 3: Cross-Shard Challenges (4 points):

Task 3.1: Cross-Shard Query Attempt (2 pts)

Try to calculate **total revenue by state** across both shards. Because shards are independent databases, you cannot JOIN across them, you must query each shard separately.

Run on the Southeast shard:

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec sales-shard-se psql
 -U student -d sales
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.

sales=# -- Run on sales-shard-se
SELECT c.state, SUM(o.total_price) AS total_revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.state
ORDER BY total_revenue DESC;
 state | total_revenue
-------+---------------
 FL    |       7291.80
 GA    |       1863.91
 TN    |        549.96
(3 rows)

sales=#
```

Run on the Northeast shard:

```
PS C:\Users\karth\Documents\ism6562-Midterm_project_BigQuest_Query\midterm\part02> docker compose exec sales-shard-ne psql
 -U student -d sales
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.

sales=# -- Run on sales-shard-ne
SELECT c.state, SUM(o.total_price) AS total_revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.state
ORDER BY total_revenue DESC;
 state | total_revenue
-------+---------------
 NC    |       1979.96
(1 row)

sales=#
Ln 263, Col 1   Spaces: 4   UTF-8   CRLF   { } Python   Finish Setup   Python 3.14 (64-bit)
```

The revenue-by-state query was executed separately on each shard because the database is horizontally partitioned. Each shard stores only a portion of the total data, so running the query on a single shard produces incomplete results. For example, the Southeast shard contains data for FL, GA, and TN, while the Northeast shard contains data for NC. To obtain a complete and accurate revenue analysis, the query must be executed on all shards and the results must be combined. This highlights a key limitation of sharded systems, where cross-shard aggregation is required for global analytics, and single-node queries cannot provide a full view of the data.

Task 3.2: Sharding Trade-off Analysis (2 pts)
Written Response (200–250 words): Discuss the benefits and drawbacks of the sharding approach used in this assignment. Address the following in your response:

The sharding approach used in this assignment partitions data across multiple databases based on geographic regions, which offers several advantages. One key benefit is geographic data locality, where data is stored closer to users, reducing latency and improving query performance. Additionally, each shard can be scaled independently based on workload demands, allowing efficient resource utilization. Fault isolation is another advantage; if one shard fails, the other shards continue to operate, improving system availability. Smaller table sizes within each shard also lead to faster query execution and reduced indexing overhead.

However, this approach introduces several challenges. Cross-shard queries require application-level aggregation, as seen in the revenue analysis where results from multiple shards must be combined manually. Maintaining consistency across reference tables, such as products, becomes complex and requires synchronization mechanisms. ETL pipelines are also more complicated because data must be extracted from multiple sources and merged before loading into the warehouse. Furthermore, uneven data distribution can create hot shards, where one shard handles significantly more traffic than others, impacting performance. Finally, sharded systems do not support cross-shard foreign keys or transactions, making data integrity enforcement more difficult across distributed databases.

Part 4: EXPLAIN ANALYZE (4 points)

```
-----------------------------------------------------------------------------------------------------------------------
--------
 Sort  (cost=53.21..53.48 rows=110 width=258) (actual time=0.129..0.141 rows=3 loops=1)
   Sort Key: (sum(o.total_price)) DESC
   Sort Method: quicksort  Memory: 25kB
   ->  HashAggregate  (cost=48.10..49.48 rows=110 width=258) (actual time=0.073..0.086 rows=3 loops=1)
         Group Key: p.category
         Batches: 1  Memory Usage: 24kB
         ->  Hash Join  (cost=12.47..38.58 rows=1270 width=234) (actual time=0.031..0.048 rows=25 loops=1)
               Hash Cond: (o.product_id = p.product_id)
               ->  Seq Scan on orders o  (cost=0.00..22.70 rows=1270 width=20) (actual time=0.007..0.009 rows=25 loops=1)
               ->  Hash  (cost=11.10..11.10 rows=110 width=222) (actual time=0.011..0.011 rows=10 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     ->  Seq Scan on products p  (cost=0.00..11.10 rows=110 width=222) (actual time=0.004..0.005 rows=10 l
oops=1)
 Planning Time: 2.287 ms
 Execution Time: 0.303 ms
(14 rows)

~
~
~
~
~
~
~
~
~
~
~
(END)Cancel request sent
(END)
```

Task 4.2: Query Plan on the Warehouse (2 pts)

Run EXPLAIN ANALYZE on the same warehouse query from Part 1:

```
-----------------------------------------------------------------------------------------------------------------------
-----------
 Sort  (cost=47.16..47.43 rows=110 width=258) (actual time=0.179..0.181 rows=3 loops=1)
   Sort Key: (sum(f.total_price)) DESC
   Sort Method: quicksort  Memory: 25kB
   ->  HashAggregate  (cost=42.05..43.43 rows=110 width=258) (actual time=0.130..0.133 rows=3 loops=1)
         Group Key: p.category
         Batches: 1  Memory Usage: 24kB
         ->  Hash Join  (cost=12.47..34.78 rows=970 width=234) (actual time=0.086..0.094 rows=28 loops=1)
               Hash Cond: (f.product_key = p.product_key)
               ->  Seq Scan on fact_sales f  (cost=0.00..19.70 rows=970 width=20) (actual time=0.013..0.015 rows=28 loops=
1)
               ->  Hash  (cost=11.10..11.10 rows=110 width=222) (actual time=0.059..0.060 rows=10 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     ->  Seq Scan on dim_product p  (cost=0.00..11.10 rows=110 width=222) (actual time=0.051..0.052 rows=1
0 loops=1)
 Planning Time: 5.694 ms
 Execution Time: 0.393 ms
(14 rows)

(END)
```

The warehouse query plan is unaffected by operational sharding because the ETL process consolidates data from all shards into a single centralized schema. In the operational system, data is distributed across multiple shards such as sales-shard-se and sales-shard-ne, which requires queries to be executed separately on each shard and results to be combined manually. However, the ETL pipeline extracts data from all shards, transforms it, and loads it into a unified star schema in the warehouse.

As a result, the warehouse contains a complete and integrated dataset, allowing analytical queries to operate on a single source without needing to consider how the data was originally partitioned. The EXPLAIN ANALYZE output reflects this, as the query performs a straightforward sequential scan on the fact_sales table, joins with the dim_product table using a hash join, aggregates results using a hash aggregate, and sorts the output. There is no indication of distributed processing or multiple data sources.

This demonstrates a key benefit of the ETL and data warehouse architecture: it abstracts away the complexity of operational sharding. Analysts can run queries efficiently without handling cross-shard logic, enabling simpler query design, improved performance, and more reliable analytical insights.